

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – Comparative Analysis

Course Code:	CSA0309	Course Title:	Data Structures
CO Assessed:	CO4 – Articulation (BL3)	Assessment:	Assessment Tool 3 – Comparative Analysis
Weightage:	30% of CIA	Total Marks:	100
CO4 Statement:	Apply hashing strategies for efficient data storage and sorting algorithms for arrangement of data.		
Student Name:	v-Yeshwanth	Reg. No.:	192511300
Section / Batch:		Date:	28/08/2026

Code of Conduct

I v-Yeshwanth (192511300) (Name / Reg No)
certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: v-Yeshwanth

Instructions

- This test contains 5 Comparative Analysis questions. Total: 100 Marks.
- When a student answer using comparative analysis should be based on four requirements: time complexity, space complexity advantages & limitations, real-world scenarios and operations.
- No negative marking. Unanswered questions carry zero marks.
- No DTP printouts or electronic devices permitted. They should provide written materials.

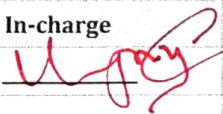
Section / Topic	Focus	Q Nos	Marks
Hashing	Linear Probing & Separate Chaining	1	20
Sorting	Merge Sort & Quick Sort	2	20
Hashing	Quadratic Probing and Double Hashing	3	20
Sorting Algorithms	Complexity Analysis	4	20
Quick Sort	Recursive and Iterative implementations	5	20
GRAND TOTAL:			100 Marks

Annexure A – Comparative Analysis

1. Compare Linear Probing and Separate Chaining for collision handling in hashing based on: (20 Marks)
 - a) Clustering effect
 - b) Memory usage
 - c) Performance under high load factor
 - d) Which method is more suitable for large-scale applications and why?
2. Compare quick sort and merge sort for sorting a large linked list of 10 million nodes. Analyze memory requirements (in-place vs. extra space), cache behavior, and time complexity. Which algorithm is preferable for linked lists and why? How does the answer change for arrays? (20 Marks)
3. Compare Quadratic Probing and Double Hashing in terms of: (20 Marks)
 - a) Clustering issues
 - b) Collision resolution efficiency
 - c) Suitability for dense hash tables
4. Compare Best-case and Worst-case time complexity of sorting algorithms based on the following: (20 Marks)
 - a) Practical significance
 - b) Impact on algorithm selection
5. Compare Recursive and Iterative implementations of Quick Sort in terms of: (20 Marks)
 - a) Space complexity (stack usage)
 - b) Ease of implementation
 - c) Risk of stack overflow

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Concept Understanding	20	Demonstrates complete and accurate understanding of all data structures with advanced insights	Shows clear understanding with minor gaps	Basic understanding; some misconceptions present	Limited or incorrect understanding
Comparative Analysis Depth	20	Thorough, multi-dimensional comparison with strong justification and critical evaluation	Good comparison with relevant factors considered	Limited comparison; lacks depth or justification	Minimal or incorrect comparison
Use of Parameters (Time/Space/Operations)	30	All parameters analysed accurately with complexity discussion	Most parameters covered correctly	Few parameters considered; some inaccuracies	Parameters missing or incorrect
Critical Thinking	20	Strong justification of why one structure is better in given contexts	Some justification present	Limited reasoning	No critical reasoning
Clarity	10	Exceptionally well-structured, logical flow, clear tables/diagrams	Well-organized with minor issues	Some organization issues; lacks clarity	Poor structure and readability

Faculty In-charge	Course Coordinator	Program Director
Signature: 	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Linear probing vs separate chaining.

Factor	Linear probing	Separate chaining
Clustering effect	Suffer from Primary clustering	No primary clustering
Memory usage.	Less memory (stores in table only)	More memory (extra linked list)
High load factor per table	Performance decreases & replay.	Handles high load factor better
Large-scale applications	Suitable when memory is limited.	Better for large database due to efficient collision handling.

2. Quick sort vs merge sort for a linked list preferred Algorithm: merge sort.

Reason:

Linked lists do not support random access, making quick sort inefficient.

merge sort divides and merges linked list easily.

Time complexity: $O(n \log n)$.

Require only pointer manipulation, no extra splitting.

For Arrays:

Quick sort is often preferred due to better
cane performance and in place sorting.

Average time complexity: $O(n \log n)$.

3.

Quadratic probing vs Double Hashing.

Factor	Quadratic probing	Double Hashing
clustering issues.	Reduces primary clustering due. has secondary clustering.	minimise both primary and secondary. clustering.
Collision resolution efficiency	Better than linear probing.	more efficiency and uniform distribution
Dense Hash. table.	performance. degrades	performs better in dense tables

Double hashing is generally surprise

Best - case and worst - case time complexity of sorting Algorithm.

Algorithm	Best case	worst case.
Bubblesort	$O(n^2)$	$O(n^2)$
Selection Sort	$O(n^2)$	$O(n^2)$
Insertion Sort	$O(n^2)$	$O(n^2)$
merge. sort.	$O(n \log n)$	$O(n \log n)$
quick Sort	$O(n \log n)$	$O(n^2)$
Heap Sort	$O(n \log n)$	$O(n \log n)$

(a). Practical significance:

Helps predict execution time.

useful for choosing suitable algorithm.

b.) Impact on Algorithm Selection

- * large datasets $\rightarrow$ merge sort, quicksort, heap sort,
- * Small or nearly sorted datasets $\rightarrow$ Insertion Sort.

Recursive vs Iterative quick sort.

Factor	Recursive quick sort	Iterative quick sort.
Space Complexity.	$O(\log n)$ recursive stack	$O(\log n)$ explicit stack
Case of implementation	Simple and readable.	more complex
Risk of stack overflow	High for large inputs	Lower risk.
Conclusion.		

Recursive sort is easier to implement while iterative sort is safer for very large datasets

Ujjay